

Digitalization and Frequency Analysis of a Voice Signal Using Python and Librosa

Authors: Alejandro Garcia Castro and Samuel Riaño Patiño

Introduction

This project encouraged us to understand how an analog voice signal can be converted into a digital one and how it can be analyzed with basic signal processing techniques in python. The idea is to take a real voice recording and go through the entire process: choosing a sampling frequency, quantizing the signal, coding it in binary, filtering it and finally analyzing its frequency content to estimate who is speaking. This kind of process is used in real applications like telephony, where voice doesn't need to keep all its frequency range to be understandable, it just needs the frequencies that make it intelligible.

General objective

- To understand and apply the process of converting an analog voice signal into a digital representation, and to analyze its frequency characteristics using python.

Specific objectives

- To justify an adequate sampling frequency for the voice signal based on the Nyquist theorem.

- To apply quantization and PCM coding so the signal can be represented in binary form.
- To design and apply low-pass, high-pass and band-pass filters over the signal.
- To analyze the frequency spectrum of the signal and estimate the fundamental frequency (F_0) in order to classify the type of speaker.

Methodology

The process was done in python using the library librosa, together with numpy, matplotlib and scipy for the filters and the FFT. First the audio file was loaded with librosa.load, that also gives the sample rate of the original recording. From there, the silent parts of the recording were removed with librosa.effects.split, so the analysis was only done over the parts where there is actually voice, this is important because the silence doesn't add any useful information and can affect the frequency detection later.

For the sampling frequency we used the Nyquist theorem, which says that the sampling frequency must be at least double the maximum frequency that we want to capture. Since the useful energy of the human voice (the fundamental frequency and the first formants) is concentrated up to around 4000 Hz, we chose a sampling frequency of 8000 Hz.

After that, three Butterworth filters were designed with a general function (butter_filter), that receives the cutoff frequency (or a list of two frequencies for the band-pass), the sampling rate and the type of filter. With this function we get a low-pass filter (under 4 kHz), a high-pass filter (over 4kHz) and a band-pass filter (between 30 Hz and 10000 Hz), and the low-pass one is the one that is more relevant for voice, because is the one that keeps the range where most of the energy is.

The signal was also denoised in a simple way, taking the average magnitude of the first frames of the STFT as a "noise profile" and subtracting it from the whole spectrogram, so the parts of the spectrum that are just noise get reduced.

For the quantization, the resampled signal (at 8000 Hz) gets normalized between -1 and 1 and then multiplied by the maximum amplitude that 8 bits can represent and rounded to the closest integer. This turns the continuous signal into a signal with just a limited number of discrete levels, which is why in the graph you can see the "steps" when compared with the original continuous signal.

Once the signal is quantized, it gets coded into binary, this is the "coding" part of the digitalization process, each sample is converted into an 8 bit binary number using two's complement, which is basically how the audio would be represented inside of a real PCM stream.

Finally, for the frequency analysis, a normal FFT was applied to obtain the frequency spectrum of the signal without silence, and additionally the pyin algorithm from librosa was used to estimate the fundamental frequency (F0) frame by frame. The median of the valid F0 values is used as the "dominant frequency", but in case pyin doesn't detect any voiced frame, the program falls back to just taking the frequency with the highest amplitude in the FFT. With that dominant frequency, the speaker gets classified in three possible categories: adult man (under 165 Hz), adult woman (between 165 and 255 Hz) or child (above 255 Hz), these thresholds are commonly used approximations of the typical F0 ranges for each group.

Results

Because the script depends on the specific audio file that gets loaded, the exact numeric values change every time it runs, but the type of results that the program gives are always the same:

- A plot comparing the original signal and the signal filtered with the low-pass filter, where it can be seen how the higher frequencies get attenuated.
- A plot comparing the normalized (continuous) signal against the quantized one, where the discrete steps caused by the 8 bit quantization are visible.
- A plot comparing the waveform of the original recording against the one with the silences removed, that is normally shorter in time because the silent parts were cut off.
- A frequency spectrum plot in dBm, with the dominant frequency marked with a vertical line and the range of the detected F0 values shaded, that also shows in the legend the classification of the speaker (man, woman or child).

Besides the plots, the console also prints the justification of the sampling frequency, the first quantized samples with their respective binary code, the method that was used to get the dominant frequency (pyin or FFT peak, depending if pyin found voiced frames) and the numeric range of the F0 detected.

Link with the video explanation uploaded to drive:

<https://drive.google.com/drive/folders/1skzGedccUiyA3NC2xSDTvB-22heKPcS>

GitHub with the different files and results:
<https://github.com/Samuel-RP22/Project-2-Sound>

Conclusion

- The accuracy of the whole process depends a lot on the quality of the recording, if there is too much background noise the pyin algorithm might not detect voiced frames and the program has to fall back to the FFT peak, which is a less precise way of finding the dominant frequency.
- Choosing 8000 Hz as sampling frequency is enough to keep the voice intelligible, but it also means that a lot of the higher frequency detail of the original recording gets lost, so there is a tradeoff between saving space (less bits, less samples) and keeping the full fidelity of the sound.
- Reducing the bit depth to 8 bits also reduces the size of the audio, but each quantized sample loses precision, which is the reason why the quantized signal looks like "steps" instead of a smooth curve when compared with the normalized one.